Universidade do Algarve

Faculdade de Ciências e Tecnologia

Licenciatura em Engenharia Informática

Redes de Computadores I

Laboratório 6

UDP e TCP em Python

Realizado por: Diogo Carvalho nº 90247

PL: 2

Docente: Rui Filipe Rosa Paulo

Ano Letivo: 2025/2026

Relatório

UDP

UDPServer.py

```
from socket import *

# Porto onde o servidor ficará à escuta.
serverPort = 12000

# Criação de um socket UDP sobre IPv4.
serverSocket = socket(AF_INET, SOCK_DGRAM)

# Associação do socket ao porto definido.
# A string vazia indica que o servidor aceita datagramas
# dirigidos a qualquer interface da máquina.
serverSocket.bind(('', serverPort))

# Mensagem informativa de arranque do servidor.
print("The server is ready to receive")

# Ciclo infinito para atendimento contínuo de pedidos.
while 1:
    # Receção de uma mensagem enviada por um cliente.
    # São obtidos os dados e o endereço do emissor.
    message, clientAddress = serverSocket.recvfrom(2048)

    # Conversão da mensagem para letras maiúsculas.
```

```
    modifiedMessage = message.upper()

    # Envio da resposta para o mesmo cliente.
    serverSocket.sendto(modifiedMessage, clientAddress)
```

UDPClient.py

```
from socket import *

# Endereço IP do servidor.
# Como o teste é feito na mesma máquina, utiliza-se localhost.
serverName = '127.0.0.1'

# Porto de atendimento do servidor UDP.
serverPort = 12000

# Criação de um socket UDP sobre IPv4.
clientSocket = socket(AF_INET, SOCK_DGRAM)

# Leitura da mensagem introduzida pelo utilizador.
message = input('Input lowercase sentence:')

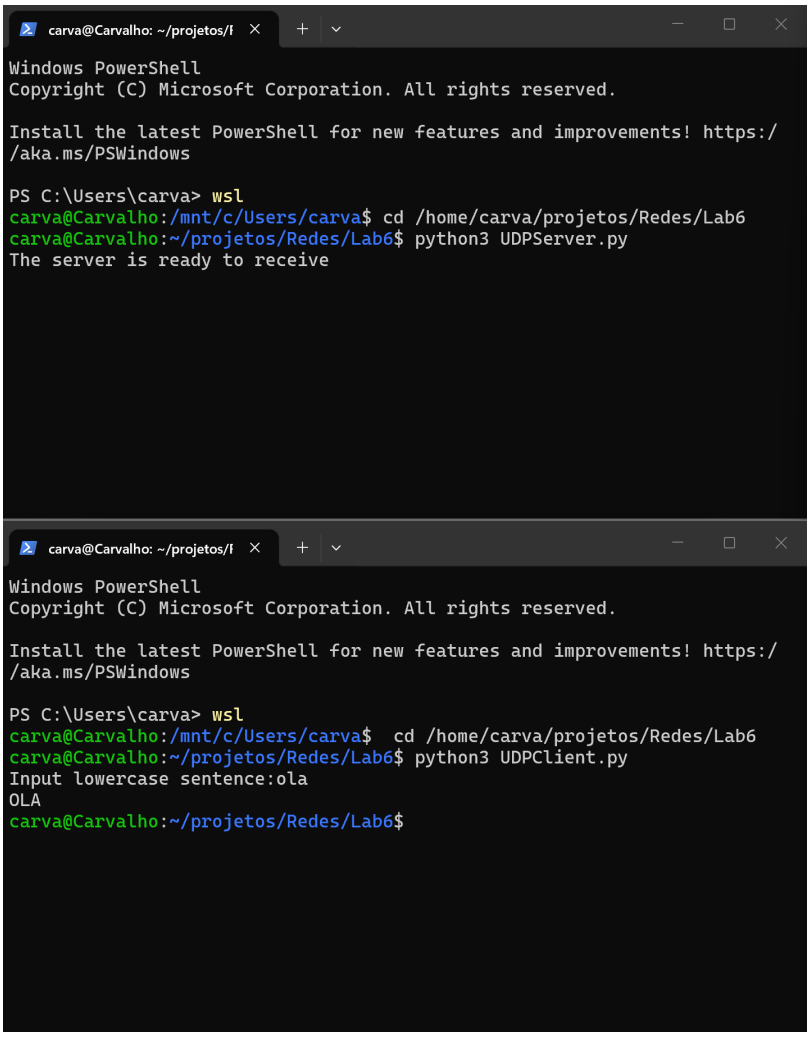
# Envio da mensagem para o servidor.
# Em Python 3, a string é convertida para bytes com encode().
clientSocket.sendto(message.encode(), (serverName, serverPort))

# Receção da resposta enviada pelo servidor.
# O método devolve a mensagem recebida e o endereço de origem.
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
```

```
# Apresentação da resposta no ecrã.
# Os bytes recebidos são convertidos novamente para string.
print(modifiedMessage.decode())

# Encerramento do socket do cliente.
clientSocket.close()
```

Output



Resumo

O programa UDP implementa uma comunicação cliente-servidor sem ligação prévia. O cliente cria um socket UDP, lê uma mensagem introduzida pelo utilizador e envia-a para o servidor através do método sendto(), indicando o endereço IP e o porto de destino. O servidor, por sua vez, fica à escuta no porto definido, recebe a mensagem com recvfrom(), converte o conteúdo para letras maiúsculas e devolve a resposta ao cliente. Finalmente, o cliente recebe essa resposta e apresenta-a no ecrã.

TCP

TCPServer.py

```
from socket import *


# Porto onde o servidor ficará à escuta.

serverPort = 12000


# Criação de um socket TCP sobre IPv4.

serverSocket = socket(AF_INET, SOCK_STREAM)


# Associação do socket ao porto definido.

serverSocket.bind(('', serverPort))


# Colocação do socket em modo de escuta.

# O argumento 1 indica a dimensão máxima da fila de ligações pendentes.

serverSocket.listen(1)


# Mensagem informativa de arranque do servidor.

print('The server is ready to receive')
```

```
# Ciclo infinito para atendimento contínuo de clientes.
while 1:
    # Aceitação de uma nova ligação.
    # É criado um socket específico para comunicar com esse cliente.
    connectionSocket, addr = serverSocket.accept()

    # Receção da mensagem enviada pelo cliente.
    sentence = connectionSocket.recv(1024)

    # Conversão da mensagem para letras maiúsculas.
    capitalizedSentence = sentence.upper()

    # Envio da resposta ao cliente.
    connectionSocket.send(capitalizedSentence)

    # Encerramento da ligação com o cliente atual.
    connectionSocket.close()
```

TCPClient.py

```
from socket import *

# Endereço IP do servidor.
# Como o teste é local, utiliza-se localhost.
serverName = '127.0.0.1'

# Porto de atendimento do servidor TCP.
serverPort = 12000
```

```
# Criação de um socket TCP sobre IPv4.
clientSocket = socket(AF_INET, SOCK_STREAM)

# Estabelecimento da ligação ao servidor.
clientSocket.connect((serverName, serverPort))

# Leitura da mensagem introduzida pelo utilizador.
sentence = input('Input lowercase sentence:')

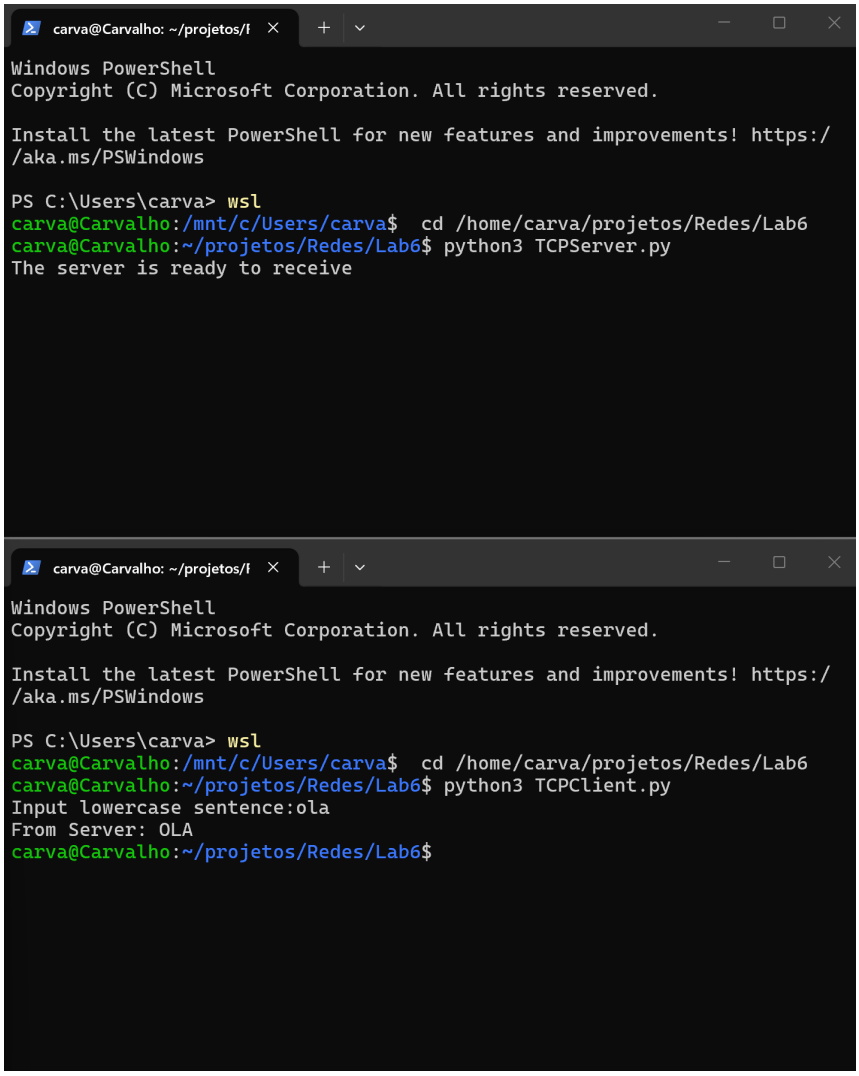
# Envio da mensagem pela ligação TCP.
# Em Python 3, a string é convertida para bytes.
clientSocket.send(sentence.encode())

# Receção da resposta enviada pelo servidor.
modifiedSentence = clientSocket.recv(1024)

# Apresentação da resposta no ecrã.
print('From Server:', modifiedSentence.decode())

# Encerramento da ligação TCP.
clientSocket.close()
```

Output



Resumo

O programa TCP implementa uma comunicação cliente-servidor orientada à ligação. O cliente cria um socket TCP e estabelece primeiro uma ligação ao servidor através do método connect(). Depois, envia a mensagem introduzida pelo utilizador. O servidor encontra-se à escuta no porto definido, aceita a ligação com accept(), recebe a mensagem, converte-a para letras maiúsculas e envia a resposta ao cliente. No final, o cliente recebe a resposta e apresenta-a no ecrã.

Conclusão/Comparação entre UDP e TCP

Em ambos os casos, o objetivo do programa foi atingido, uma vez que o servidor recebeu a mensagem enviada pelo cliente, converteu-a para maiúsculas e devolveu a resposta corretamente.

A principal diferença entre UDP e TCP está na forma como a comunicação é realizada. No UDP não existe ligação prévia entre cliente e servidor, sendo a mensagem enviada diretamente através de datagramas. No TCP, pelo contrário, é necessário estabelecer primeiro uma ligação antes da troca de dados.

Assim, conclui-se que o UDP apresenta uma comunicação mais simples e direta, enquanto o TCP utiliza um mecanismo de comunicação orientado à ligação.

